

RESUMO - STORED PROCEDURES, TRIGGERS E TRANSAÇÕES

1. STORED PROCEDURE

O que é?

Uma Stored Procedure é um conjunto de comandos SQL armazenado no banco de dados que pode ser executado posteriormente através do comando:

```
CALL nome_da_procedure();
```

Ela funciona de forma semelhante a uma função.

Estrutura Básica

```
DELIMITER //
```

```
CREATE PROCEDURE sp_exemplo()  
BEGIN  
    SELECT 'Olá';  
END//
```

```
DELIMITER ;
```

Execução:

```
CALL sp_exemplo();
```

BEGIN e END

Servem para delimitar o bloco de comandos da procedure.

```
BEGIN  
    comando1;  
    comando2;  
END
```

DELIMITER

Altera temporariamente o caractere que encerra comandos SQL.

Normalmente:

;

Mas como Procedures e Triggers possuem vários ";" internos, utiliza-se:

DELIMITER //

ou

DELIMITER ##

ou qualquer outro delimitador escolhido.

Após finalizar:

DELIMITER ;

Parâmetros

IN

Recebe valores.

```
CREATE PROCEDURE buscar_cliente(  
    IN p_id INT  
)
```

Execução:

```
CALL buscar_cliente(1);
```

OUT

Retorna valores.

```
CREATE PROCEDURE total_clientes(  
    OUT total INT  
)
```

INOUT

Recebe e retorna valores.

```
CREATE PROCEDURE soma10(  
    INOUT valor INT  
)
```

Exemplo:

Valor entra = 20

Procedure soma 10

Valor sai = 30

DECLARE

Cria variáveis locais.

```
DECLARE total INT;
```

Essas variáveis existem somente dentro da procedure.

IF

Permite criar condições.

```
IF saldo > 1000 THEN  
    SELECT 'VIP';  
ELSE  
    SELECT 'COMUM';  
END IF;
```

CASE

Alternativa ao IF.

```
CASE
  WHEN saldo > 1000 THEN 'VIP'
  ELSE 'COMUM'
END
```

2. TRIGGERS

O que é?

Uma Trigger é um código SQL executado automaticamente quando ocorre um evento em uma tabela.

O usuário não executa uma trigger manualmente.

Eventos que disparam Trigger

INSERT

Quando um registro é inserido.

UPDATE

Quando um registro é alterado.

DELETE

Quando um registro é removido.

BEFORE e AFTER

BEFORE

Executa antes do evento.

BEFORE INSERT

AFTER

Executa depois do evento.

AFTER UPDATE

Estrutura Básica

DELIMITER //

```
CREATE TRIGGER tr_exemplo  
BEFORE INSERT  
ON cliente  
FOR EACH ROW  
BEGIN
```

```
    -- código
```

```
END//
```

```
DELIMITER ;
```

ON tabela

Define em qual tabela a trigger atuará.

ON cliente

FOR EACH ROW

Significa:

"Executar para cada linha afetada."

Se um INSERT inserir 3 registros:

A trigger será executada 3 vezes.

NEW e OLD

NEW

Representa o valor novo.

Exemplo:

NEW.nome

OLD

Representa o valor antigo.

Exemplo:

OLD.nome

Disponibilidade

Evento	Pode usar
INSERT	NEW
UPDATE	OLD e NEW
DELETE	OLD

Exemplo

```
SET NEW.nome = UPPER(NEW.nome);
```

Transforma o nome em maiúsculo antes de salvar.

SIGNAL SQLSTATE

Utilizado para gerar erros manualmente.

Exemplo:

```
IF NEW.preco < 0 THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Preço inválido';
```

```
END IF;
```

SQLSTATE 45000

Significa:

"Erro definido pelo usuário."

É o código mais comum em exercícios e provas.

Para que serve o SIGNAL?

- Impedir operações inválidas
 - Validar regras de negócio
 - Cancelar INSERTs ou UPDATEs incorretos
-

3. TRANSAÇÕES

O que é?

Um conjunto de operações que devem ser executadas completamente ou não executadas.

Exemplo:

Transferência bancária.

START TRANSACTION

Inicia a transação.

```
START TRANSACTION;
```

COMMIT

Confirma as alterações.

```
COMMIT;
```

ROLLBACK

Desfaz as alterações.

```
ROLLBACK;
```

Exemplo

```
START TRANSACTION;
```

```
UPDATE conta  
SET saldo = saldo - 100  
WHERE id = 1;
```

```
UPDATE conta  
SET saldo = saldo + 100  
WHERE id = 2;
```

```
COMMIT;
```

Relação entre Trigger e Transação

A Trigger participa da transação.

Porém:

 Não pode usar COMMIT

✗ Não pode usar ROLLBACK

✗ Não pode iniciar transações

Se uma Trigger gerar erro usando SIGNAL, a operação é cancelada.

DIFERENÇA ENTRE PROCEDURE E TRIGGER

Procedure	Trigger
Executada manualmente	Executada automaticamente
Usa CALL	Não usa CALL
Pode receber parâmetros	Não recebe parâmetros
Pode ser executada quando desejar	Depende de um evento
Automatiza tarefas	Reage a eventos

RESUMO FINAL

Procedure:

- CREATE PROCEDURE
- CALL
- BEGIN/END
- IN, OUT, INOUT
- DECLARE
- IF

Trigger:

- CREATE TRIGGER
- BEFORE ou AFTER
- INSERT, UPDATE, DELETE
- NEW e OLD
- SIGNAL SQLSTATE '45000'

Transações:

- START TRANSACTION
- COMMIT
- ROLLBACK